

J-Vision

Ben Meyers

September 2026

1 Through the Jacobian Lens

Anthropic’s July 2026 paper introduces a new causal lensing approach to interpret the cognition of Large Language Models (LLMs), featuring the **Jacobian Lens (J-lens)** and the **J-space**.

LLMs are deep neural networks trained on massive amounts of language data captured or created by humans. Deep learning delivers us artifacts that are at once extremely competent and hard to interpret. The contract that enables deep learning to deliver such artifacts is (1) legible *ends* (input/output) and (2) *end-to-end* differentiability. By configuring our desired behavior (such as predicting text from text) as a program that complies with the above contract, we invite the backpropagation algorithm to causally map the network and iteratively realize our goals through it. Training is a series of instantaneous lenses (how can *this specific error* be related to *these current parameters* and reduced in their terms?), accumulated into expectations (how does each parameter, in general bear on the *average* error and how, then, should we change them to reduce it?), which are then committed as parameter updates. Once the model has been trained based on aggregate expectation, it is used for individual, specific inputs—we do not want *inference* to be an aggregate activity, we want each distinct input to receive duly nuanced treatment under the trained model. With the **J-lens**, however, we construct an *expected* lens on how those specific instances behave in general. We trained with expectation, so now let’s reverse-engineer with it. The deep learning contract sanctions on our ability to probe the network with calculus, and to weave our local, specific probes into generalized expected probes. Figure 1 shows a matrix of four kinds of lenses endowed by the contract and used in training and interpretability.

Specifically, the **J-lens** is an expected derivative of *logits*—scores given by the model to the tokens (word fragments) at its disposal for sampling next—with respect to *activations*—the forward-propagating internal states of the model. It is a matrix, where each v -th row represents the logit index for token v in the vocabulary, and each d -th column of row v denotes the influence of activation dimension d on that v -th token. Figure 2 articulates the shape and nature of the object. Being a derivative object, it doesn’t give us static data about what the activations *stand for*, but instead where they *point*. This matrix can then

	<i>at one instance</i>	<i>in expectation</i>
w.r.t. <i>weights</i>	$\frac{\partial \text{loss}}{\partial W}$ one example's gradient	$\mathbb{E} \left[\frac{\partial \text{loss}}{\partial W} \right]$ the direction training follows
w.r.t. <i>activations</i>	$\frac{\partial \text{logits}}{\partial h}$ attribution in one context	$\mathbb{E} \left[\frac{\partial \text{logits}}{\partial h} \right]$ the J-lens

Figure 1: Four derivative objects from the same contract. The top row has always steered *training*; the bottom-left explains single predictions after the fact. The fourth square—activation Jacobians taken *in expectation*—is the new instrument.

attribute disposition—in terms of propensity toward *tokens*, which are proxies for *concepts*—to the heretofore inscrutable latent space of the model.

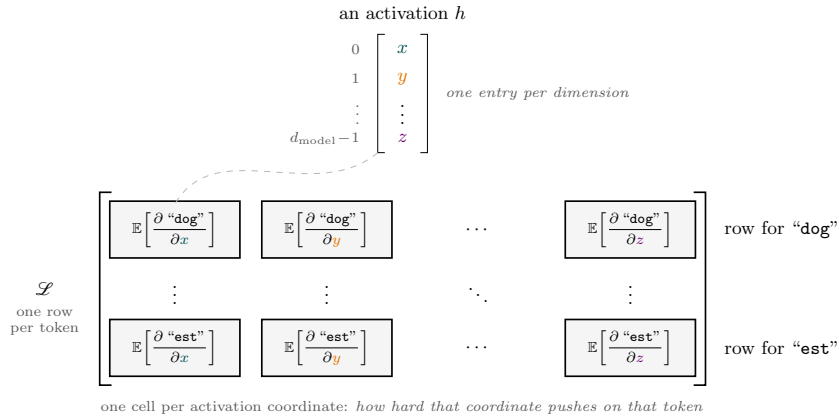


Figure 2: What the rows of the **J-lens** are. An activation h carries one number per model dimension (left, indexed $0 \dots d_{\text{model}}-1$). The **J-lens** holds one *row* per token in the vocabulary, and that row holds one entry per activation coordinate: the *expected* derivative of that token's logit with respect to that coordinate—averaged, as every entry here is, over the contexts and positions the lens was fit on. Reading across the “dog” row tells us how each direction of h pushes the model toward saying *dog*; the “est” row says the same for *est*. The matrix is thus $|V| \times d_{\text{model}}$ —as many rows as there are tokens to say, as many columns as there are dimensions to say them with.

It is important to impress that the **J-lens** doesn't just *correlate* internal activations with external verbalization potentials. The **J-lens** is only statistical to the extent that it is the product of *averaging* many precisely correct values. It does not offer a merely post-hoc, explanatory picture of interpretability, where statements can be made to the effect of: “this circuit of neurons tends to fire when the model discusses dogs, so therefore this circuit may encode something about dogs.” These networks (when trained) are entirely deterministic and static; these Jacobians speak only the language of analytically-derived *cause*. They allow us to say: “this bundle of activated neurons reflects that the model

is disposed to mention dogs; if we amplify these currently-activated neurons, the model *will* mention dogs more.”

Further, once the directions in the model’s activation space are inflected conceptually, we can do better than just *identify* disposition implied by state. We can tap the tools from the abstract formalism of linear algebra to work—observe, illustratively decompose, perturb, and steer—in a now-interpretable space with economy. Cue the **J-space**. The **J-lens** gives us white light—potent, but over-saturated and noisy. The **J-space** is a prism fit on this light, in the form of a linear decomposition. Through it, we only keep those **J-lens** directions that carry the disproportionate weight of the model’s disposition as specified by the lens.¹ The prism proves more than pretty; the **J-space** has been shown (in diverse though duly limited experiments, detailed in §5) as a privileged causal seat—which LLMs learned to use via training, and which we are now learning to use as well.

This document will explain the definitions and techniques employed to compute the **J-lens** and the **J-space** and will seek to elucidate how rewarding our chosen formulation for deep learning—*differentiable vector spaces*—is and can continue to be.

2 Transformer Architecture: Positions, Layers, Logits, the Residual Stream, and Normalization

To appreciate the **J-lens** and the **J-space**, a review of some principal features of the **Transformer** neural network architecture (that which carries out most modern LLMs) is in order. The key elements we’ll discuss are:

1. The manner in which token **Position** is represented and handled in the transformer’s computation
2. The way computation propagates forward through the **Layers** of the network
3. The key output of the network which represents its prediction for best choice of *next* token, the non-normalized vector of **Logits**
4. The cord which carries *state* through the layers of the model, the **Residual Stream**
5. **Normalization** as a mechanism of abstraction in spatial representation

¹If we can expect to see anything like cognition and thoughts inside a language model, we have to hope that they can be put in the reduced terms of a few dozen—not dozens of thousands—of concepts.

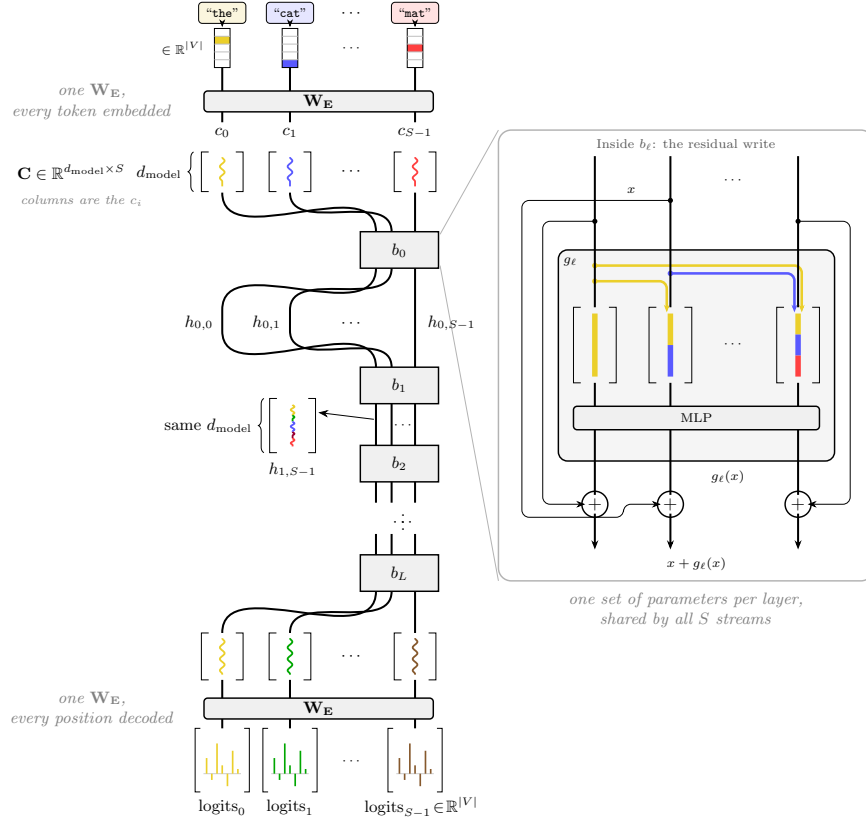


Figure 3: The transformer as one stack of blocks b_ℓ and S residual stream cords. Tokens enter as text, become one-hots, and are mapped by a single shared $\mathbf{W}_E$ into the columns c_i of $\mathbf{C}$ (top). The blocks carry and mix these residual states through depth. Every final latent is decoded by a single shared $\mathbf{W}_E$ into its logits (bottom). Colors trace content—each token enters in its own hue, attention *moves* it across positions, and depth *mizes* the bands into the final blended states.

At a high level, the transformer takes as input a context matrix, $\mathbf{C}$, where each column, c_i , represents a token of input as a vector.² The transformer admits any sequence length, S , meaning $\mathbf{C}$'s column count is *free*. The dimension of each c_i , though, is fixed model-wide at d_{model} . The network's weights contract against d_{model} , so S naturally carries forward seamlessly through each layer.

Strictly speaking, the transformer computes S final activation vectors, $h_{L,t}$, for all $t < S$.³ Though it is performed on all final activations during training, once the model is trained and ready to be used for autoregressive *generation*, only the final activation $h_{L,S-1}$ is decoded via an *unembedding* matrix, $\mathbf{W}_{\mathbf{U}} \in \mathbb{R}^{|V| \times d_{\text{model}}}$, where V is the token vocabulary and $|V|$ its size. This decoding produces a vector, $\text{logits}_{S-1} \in \mathbb{R}^{|V|}$, representing the raw weight assigned to every possible token in the vocabulary as a candidate to come *next* in the sequence. logits_{S-1} itself is not a probability distribution—it still awaits normalization via the Softmax function—but it is its direct proxy, as Softmax maintains order. Once we turn logits_{S-1} into a probability distribution via Softmax, we can sample the distribution, choose a new token, and create a new context matrix $\mathbf{C} \in \mathbb{R}^{d_{\text{model}} \times (S+1)}$ where c_S is the vector representation of the new token, and repeat.

2.1 Positions

In autoregressive generation only the final position's logits are decoded into the next token, though tokens at all positions must be fed into the model for any forward pass. Correspondingly, logits are produced at *every* position—in fact, during training, every position's logits receive a loss. Why include every prior position? The transformer architecture is such that each latent $h_{\ell,t}$ is informed by all of the tokens at positions $i \leq t$. In each transformer block, b_{ℓ} , there is a module known as an **Attention Mechanism**, which facilitates the flow of information from the previous boundary's latents $h_{\ell-1,i}$ into $h_{\ell,t}$, for all positions $i \leq t$. Figure 3 articulates this information flow at a high level.

This forward-only flow of contextual information along the token axis is a vital element of the construction of the **J-lens**. Though logits_{S-1} is produced by decoding only $h_{L,S-1}$ directly, attention makes it such that a latent $h_{\ell,t}$ at any layer and position has a causal impact on logits_{S-1} .⁴

The goal for the **J-lens** is to formalize a general relationship between any latent and all downstream logits, on average. This is precisely what enables

²This explanation of transformers ignores tokenization, one-hot representation, and embeddings and jumps right to contexts as being already embedded. Figure 3 hints at the tokenization and embedding stages, but they are less pertinent to the **J-lens** and **J-space**.

³A note on subscripts, used throughout: the first indexes the *layer boundary*— $h_{\ell,t}$ is the residual state at position t just after block b_{ℓ} , with L reserved for the final layer—and the second indexes the *position*. At various points in the document, either subscript will be omitted when it doesn't detract from what is being explained; if there is only one subscript, it corresponds to *layer*.

⁴With one notable exception: final-layer latents at non-final positions, $h_{L,t}$ with $i < S-1$, are causal dead ends for logits_{S-1} , since no attention remains downstream of them to carry their contents across streams.

it to inform *general verbalizable disposition*, as opposed to a more constrained quantity only pertaining to the logits at any latent’s own position. §3.1 covers the precise mechanism in more detail, but in computing the **J-lens**, we take Jacobians and average them over both directions of the position axis, pairing every source latent $h_{\ell,t}$ with every logit vector it feeds, $\text{logits}_{t'}$ for $t' \geq t$.⁵ The payoff is that each layer’s **J-lens** $\mathcal{L}_\ell$, once computed, is not bound by position at all—we get one $\mathbf{W}_U$ -shaped map, applicable to any latent in the stream at layer ℓ .

2.2 Layers

Whereas attention provisions for information to move forward along the *token position* axis, a transformer’s depth in *layers* allows for complex, highly nonlinear processing to occur at each position. Feedforward neural networks, in general, are compositions of functions, where each layer represents another function on the output of the layers before it. When layers are given nonlinear activation functions, their composition becomes *highly* nonlinear.

At each layer of a transformer, tokens are being processed at different levels of abstraction. For idealized instance, early layers of transformers may be focused more on composing a *syntactic* representation of the context, which, in later layers, is then processed for its *semantic* and *factual* content (syntax having been a prerequisite for anything semantic). For this reason, having a **J-lens** at every layer provides distinct insight into leverage over logits_{S-1} at different levels of abstraction. It allows us to trace disposition through depth of representation.⁶

Further, because of the compositional nature of the transformer, the layers downstream from any specific layer, ℓ , can be considered a single nested function, F_ℓ , that maps h_ℓ to logits (with the rest of the context held fixed). This framing, displayed in Figure 4, becomes insightful for the purposes of computing and conceptualizing **J-lenses**. Just as usefully, we can name the purely *internal* portion of this nesting. Let R_ℓ be the map that carries a latent h_ℓ forward through the remaining blocks to the *last* latent h_L : $R_\ell(h_\ell) = h_L$. It is all of F_ℓ except the final step that turns h_L into logits. Where F_ℓ lands in logit space ($\mathbb{R}^{|V|}$), R_ℓ lands right back in latent space ($\mathbb{R}^{d_{\text{model}}}$). The two nest in lockstep— $R_\ell \supset R_{\ell+1} \supset \dots$ mirroring $F_\ell \supset F_{\ell+1} \supset \dots$ —but no R_ℓ ever reaches that final readout, and R_L is simply the identity. Figure 4 draws both nestings at once. This internal map is the honest home of the network’s nonlinearity, and it is what our core Jacobian will differentiate.⁷

⁵For a context of $S = 100$ tokens, that is $100 + 99 + \dots + 1 = 5050$ Jacobians per layer. $h_{\ell,0}$ contributes one Jacobian for the 100 downstream logit vectors, and logits_{99} receives one from each of the 100 latents in layer ℓ that inform it. Furthermore, beyond just averaging Jacobians over token positions within a single context, the **J-lens** also averages them across contexts.

⁶In fact, Anthropic finds that the **J-space** only becomes significantly decisive in a middle band of layers.

⁷It is vital to notice the omission of the *token position* subscript on h_ℓ here. Because of attention, F_ℓ requires all of the information from prior token positions. Isolating F in our minds is just a tool to represent “the downstream network” from a given layer.

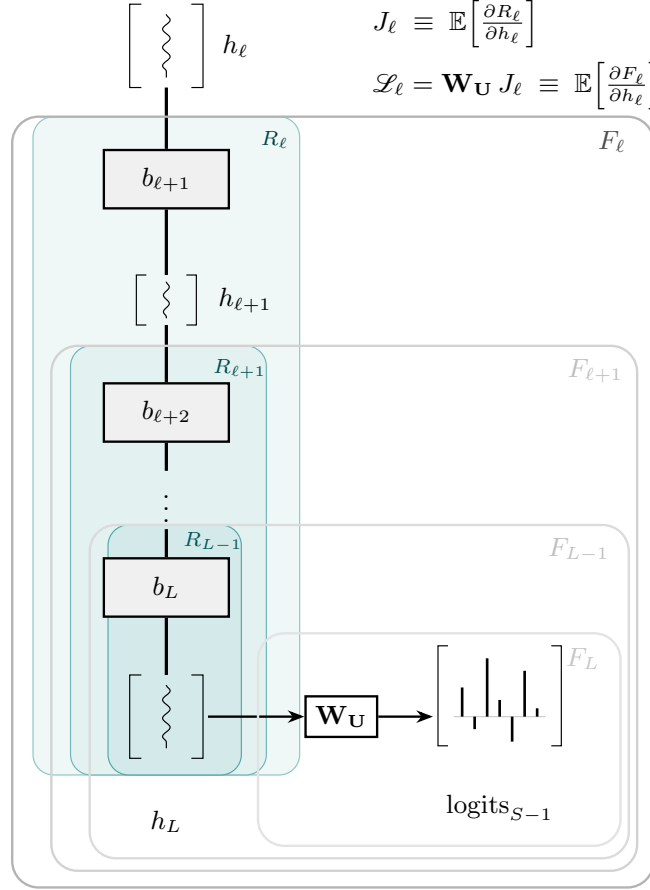


Figure 4: The nested downstream functions: from h_ℓ , everything below is F_ℓ ; one layer deeper, $F_{\ell+1} \subset F_\ell$. The **J-lens** at layer ℓ is the averaged differential of F_ℓ . Note that F_ℓ includes $\mathbf{W}_U$: the composition lands in logit space, which is why the **J-lens** shares $\mathbf{W}_U$'s shape, $|V| \times d_{\text{model}}$. The teal bands are the **transports** $R_\ell \supset R_{\ell+1} \supset \dots$ —the same nesting stripped of the readout, each carrying its latent only as far as h_L and never reaching $\mathbf{W}_U$, so each lands back in $\mathbb{R}^{d_{\text{model}}}$. The averaged differential of R_ℓ is the core Jacobian J_ℓ , and $\mathcal{L}_\ell = \mathbf{W}_U J_\ell$. The innermost nests make the telescoping explicit: F_{L-1} is b_L followed by $\mathbf{W}_U$, and F_L is nothing but $\mathbf{W}_U$ itself—there the differential is exact and context-free, and R_L is the identity; the deeper the tap, the more nonlinearity the **J-lens** must average over.

2.3 Logits

As Figure 3 and the above discussions elucidate, the transformer is trained to produce final latents, $h_{L,t}$, at all token positions ($t < S$). It is simultaneously trained to fit an unembedding matrix, $\mathbf{W}_U \in \mathbb{R}^{|V| \times d_{\text{model}}}$, which can map any of these final latents $h_{L,t}$ to logits $_t$ vectors. Logits are the non-normalized values corresponding to each token, that are collectively fed to the Softmax function to turn them into a probability distribution over tokens. In essence, $\mathbf{W}_U$ learns a mapping from the transformer’s general size for representing one token— d_{model} —to the size of the entire available vocabulary— $|V|$. Importantly, though, $\mathbf{W}_U$ was trained to expect that its input latent would contain contextual information pertaining to the entire prior context, already computed (i.e. it was trained to expect activations from the *final layer* L , not before). What we want with **J-lenses** is the ability to—in a matrix of the same *shape* as $\mathbf{W}_U$, translating across the same *spaces* as $\mathbf{W}_U$ —interpret an “unfinished” latent—that is, a latent at some non-final layer $\ell < L$ —for the logits it portends.

Prior research (the *logit lens*) has used $\mathbf{W}_U$ to directly decode non-final latents into logits; while at-times insightful, this work ignores the reality of what $\mathbf{W}_U$ has been trained to expect as input and the characteristically non-linear transformations that get us there. The **J-lens** is the principled successor to these prior methods as it faithfully differentiates back through the network, drawing a direct causal link between any latent and the ultimate logit vector it affects.

Why make the **J-lens** differentiate from logits and not from post-Softmax probabilities if those probabilities are what are ultimately used to sample the next token? While Softmax exaggerates the shape of the raw logit distribution, for the purposes of the **J-lens** and **J-space**—meant to quantify relative directions of disposition in the model’s latents—logits will suffice without the additional derivative computation through Softmax.⁸ A positive causal relationship with some logit entry monotonically translates to a positive causal relationship with the corresponding token’s probability of being sampled (all else being equal).

2.4 The Residual Stream

One of the key features of the transformer is its employment of *residual* layers, as opposed to traditional feedforward layers. A residual layer in a neural network is a function of the form $f(x) = x + g(x)$ —where g is the layer’s actual learned transformation—rather than $f(x) = g(x)$ outright. In essence, residual layers’ weights are trained not to fully transform their inputs into outputs, but instead to compute *what should be added* to their inputs to achieve some ideal output. One intuition for the plausibility of the construction is that instead of burdening the weights with the task of (a) doing some insightful computation on x and (b) carrying forward the information in x that should be kept, weights only

⁸Which, in fact, warps the confidence of the distribution, skewing the **J-lens** based on exponential confidence.

need to learn (*a*). As a direct complement to residual layers’ ability to carry information *forward* in the network, they also act as smooth highways for the *backward* flow of gradients. The residual construction provides a kind of ballast for backward-propagating derivatives to be able to directly reach ever earlier layers without exploding or vanishing through weight matrices. Differentiating the residual form makes this plain: $\frac{\partial f}{\partial x} = I + \frac{\partial g}{\partial x}$. Every residual layer’s Jacobian is the identity plus a learned correction, so the Jacobian of an entire stack of layers is a product of near-identity factors—an unbroken causal highway from any latent to any later one.

Residual layers have been successfully utilized before the transformer, but the transformer’s entire architecture embraces the construction as first class (see Figure 3). The **Residual Stream** is the conventional name attributed to the additive paths that course the computational graph of the transformer. Transformer blocks are typically referred to as “reading from” and “writing to” the residual stream, connoting the stream itself as a kind of progressive *state* of the network. Of course, you could interpret values passing from layer-to-layer of any non-residual network as representations of state, but when the entire network is emphatically trained on the residual paradigm, these residual vectors can more faithfully be understood as more than mere “intermediate” representations. They don’t just happen to look like stateful highways—training actively optimizes towards this construction.

In keeping with the conventions mentioned above, **J-lenses** tie causal links from logits back to *residual* activations—i.e. vectors at different spots in the residual stream. They, more than intermediate vectors *within* blocks, naturally represent states in the network’s forward pass. The **J-lens** then gives us interventional and observational insights into *what the model has the propensity to do* given its progressive states.

2.5 Normalization

Transformers, as we’ve seen, carry state forward via the residual stream. Latents are read into blocks, where they’re transformed with tensor operations—most of which are dot products—and written back to the stream for the next layer. Within blocks, these vectors are copiously normalized,⁹ stripped of their excess magnitude and kept for just the *direction* they point in space (or, equivalently, the *angles* they represent along the axes of high-dimensional spheres).

Why normalize? Why *direction* over precise *location*? For one thing, rampant normalization makes training much smoother as the magnitude of gradients—the strings by which we puppeteer parameters into proper settings—are kept always within a tamed range. Data becomes relative, not absolute and parameters become decision-makers, not reinforcers. There are several interpretations for *vectors*. As *points*, they can represent locations and their differences are lengths or distances. As *arrows*, they can represent directions and their differences are angles. Both angles and lengths can represent with specificity, but

⁹Either at the beginning of blocks, at the end, or both, depending on implementation.

angles come “out of the box” with several distinct advantages for the purposes of deep networks.

For systems we want to be generally decisive, capable of abstraction, and resistant to over-fitting memorization, direction and angle are a natural choice over location and distance. The former is *bounded*, *signed*, and *scale-free*. It represents *qualities* quantitatively, the exact mission of networks that we want to work in the space of concepts and categories. *Alignment*, *indifference*, and *opposition* are the language of the normalized dot product as +1, 0, and −1, respectively.¹⁰ How can you express these notions with location and distance?

$\mathcal{L}$ and $\mathcal{S}$ are fundamentally about direction and alignment. As we’ll see in §4, states in h -space are more decisively decomposed into their *distinct* concepts—(near-)orthogonal directions are the only way to represent this notion.

In the architectural hyperparameters of a network, we can *declare* what training will *instill* as first-class paradigms for computation. We did this with the residual stream and we continue it with normalization. Normalization of vectors on their way in and of residual adjustments on their way out of blocks enables the *directional* paradigm to become a privileged logic in the transformer’s computation. In fact, the paradigms of residuals and normalization go hand in hand. Understanding representation as arrows, the *additions* that blocks make to the residual stream become angular nudges after the normalization that starts off every next block. b_ℓ adds a nudge-vector to the tail of h_ℓ , and $b_{\ell+1}$ immediately grabs the hypotenuse and shrinks it back to standard size, realizing $b_\ell(h_{\ell-1})$ as a *rotation* before adding its own.

Wait, if the additions contributed by residual layers are only fully realized as rotations in the *subsequent* layer, does this undermine the notion that a latent vector in the residual stream h_ℓ represents state? In one sense, yes, it provides us due skepticism that any h_ℓ is a ‘perfect’ representation at any point.¹¹ Recall though, the defining improvement that $\mathcal{L}$ makes upon the *logit lens*: it cuts no corners because it differentiates *through* the downstream network! Reading off h_ℓ directly would be mistaken; but reading it through $\mathcal{L}_\ell$ —which accounts for the normalization in $b_{\ell+1}$ as part of its reading—honors exactly what we want.

3 J-lens

The **J-lens** and its family are the key objects per layer that allow all of the analysis that follows. At the core of the family is $J_\ell \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, the expected Jacobian of the final latent h_L with respect to a prior latent h_ℓ .

$$J_\ell = \mathbb{E}_{t' \geq t, \mathcal{D}} \left[\frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \right]$$

¹⁰Normalized dot products, not dot products in general. Without normalization, direction and magnitude are conflated, allowing two vectors to score high in a dot product because of either true alignment or just a large length.

¹¹We shouldn’t have had that view in the first place, though, as opening a latent variable is by-definition seeking an unfinished value.

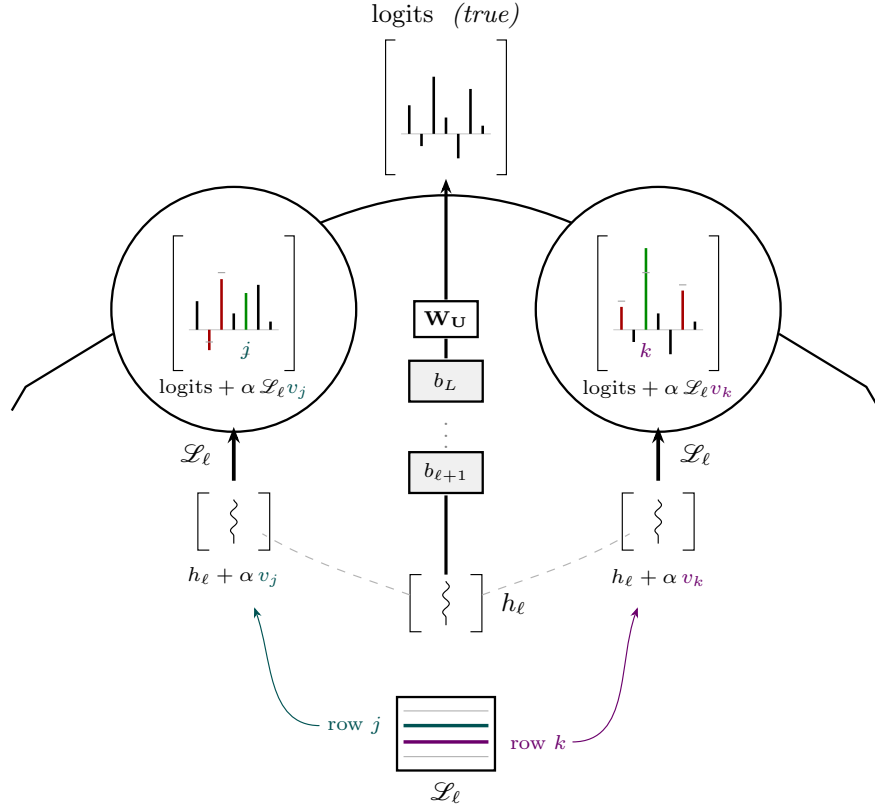


Figure 5: Counterfactual vision *through the J-lens*: without rerunning the network, we can attain approximations of where logits will go with specific nudges on h_ℓ . The **J-lens** enables this vision. The true state h_ℓ (center) earns its logits the hard way—up through every remaining block and $\mathbf{W}_U$. The flanking states are tweaked copies of h_ℓ , each nudged along a row of the J-lens $\mathcal{L}_\ell$ itself (rows j and k , drawn out of the matrix at bottom)—and they never run the network at all: through the glass, $\Delta \text{logits} = \mathcal{L}_\ell \Delta h$, one matrix multiply. Green bars were raised by the tweak, red lowered; gray ticks mark the true heights. Each eye’s tall green bar sits exactly at the token whose row it was steered with.

While the details of the *expectation* (over token positions t , over prompt corpus $\mathcal{P}$) will be elucidated in due course, the thrust of this object is a representation of how much some downstream final-layer latent $h_{L,t'}$ is sensitive to an upstream latent $h_{\ell,t}$ —where “downstream” here refers to both of the transformer’s causal axes: *token position* and *layer*. Setting aside token positions momentarily, we have:

$$J_\ell \equiv \mathbb{E}_{t' \geq t, \mathcal{P}} \left[\frac{\partial R_\ell}{\partial h_\ell} \right]$$

This is the core step skipped by the *logit lens*—the differentiation through the nonlinear layers of the network that make a latent ready for decoding via $\mathbf{W}_U$.¹²

The natural next step to get these latents speaking about *logits* is to compose J_ℓ with $\mathbf{W}_U$. This gives us the **J-lens** $\mathcal{L}$, which shares the shape (and thus the structure of translation) with $\mathbf{W}_U \in \mathbb{R}^{|V| \times d_{\text{model}}}$.

$$\mathcal{L}_\ell = \mathbf{W}_U \cdot J_\ell$$

This matrix represents the sensitivity of some downstream logits $_{t'}$ to an upstream latent $h_{\ell,t}$. It tells us, in each row $v \in \mathbb{R}^{d_{\text{model}}}$, the gradient of logits $_{t'}[v]$ with respect to $h_{\ell,t}$. These rows, in essence, represent the directions we’d want to push $h_{\ell,t}$ in order to make logits $_{t'}[v]$ higher (i.e. to make token v more likely to be sampled downstream). If we take activations $h_{\ell,t}$ to be *arrows denoting direction* as opposed to *points denoting location*, we can then measure the *alignment* between the current direction of $h_{\ell,t}$ and that of steepest logit-increase for any token we please. This brings us to the normalized **J-lens** $\mathcal{L}(h)$ *function*.

$$\mathcal{L}_\ell(h_\ell) = \text{Softmax}(\mathbf{W}_U \cdot \text{norm}(J_\ell \cdot h_\ell))$$

This takes us from some $h_{\ell,t}$ to a probability distribution ($\mathbb{R}^{|V|}$) of the model’s propensity to verbalize each token downstream.¹³ $\mathcal{L}_\ell(h_\ell)[v]$ tells us the expected odds of verbalization of token v , as indicated by the normalized, expected Jacobian of future latents $h_{L,t'}$ with respect to $h_{\ell,t}$. Figure 5 pumps intuition about the relationships J_ℓ spells out and how they can be used for counterfactual insight (then, factual interventional success).

As the computation below will detail, any J_ℓ where $\ell < L$ is but a *local linear approximation* of a relationship that is by no means linear. Further, J_ℓ as we define it is an *average* local linear approximation of the relationship in question. Any from the set of specific Jacobians averaged to produce a single object from this family is itself entirely accurate and complete, but only to the specific $h_{\ell,t}$ that allowed its computation. R_ℓ is (for $\ell < L$) by-design nonlinear, so the

¹²If the *logit lens* used $J_\ell \cdot h_\ell$ instead of just h_ℓ , it would just about be the **J-lens** $\mathcal{L}_\ell$.

¹³If we did not want to normalize this vector of alignment scores into a probability distribution, we’d still want to normalize $J_\ell \cdot h_\ell$ because $\mathbf{W}_U$ expects normalized latents, and we’ve explicitly taken J_ℓ only through the last latent, leaving out the network’s typical normalization thereof.

transformation it performs on $h_{\ell,t}$ is *dependent on $h_{\ell,t}$* and irreducible over h_{ℓ} -space without averaging.¹⁴ Now the averaging, for all the specific accuracy it sheds, makes J_{ℓ} and $\mathcal{L}_{\ell}$ flexible over token positions and—when further averaged over the prompt corpus $\mathcal{P}$ —flexibly relevant across semantic contexts.

Finally, as §4 will show, the **J-space** decomposition $\mathcal{S}_{\ell}(h_{\ell})$ of a latent h_{ℓ} is a selection of the rows of $\mathcal{L}_{\ell}$. Thus, J_{ℓ} and $\mathcal{L}_{\ell}$ are the prerequisite objects for **J-vision** broadly.

3.1 Computation

Our first primitive is the Jacobian $\frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ for any specific t and $t' \geq t$. This matrix is built iteratively in d_{model} steps. Having cached the activations of a forward pass, we can compute the local gradient at each layer for the specific input. In order to unambiguously represent the effective $\mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{model}}}$ matrix that took $h_{\ell,t}$ to $h_{L,t'}$ we have to individually probe the pass in each dimension with unit vectors. When $h_{\ell,t}$ went through this transformation in the forward pass, it collapsed and lost the information of the maps at work on it in the process through frequent weighed summation. To recover the collapsed information, we seed d_{model} forward passes from layer ℓ (where each layer’s “weights” are local Jacobians, frozen at the cached activations of $h_{\ell,t}$ ’s forward pass) with each standard basis vector e_d of $\mathbb{R}^{d_{\text{model}}}$ to get each d -th column of $\frac{\partial h_{L,t'}}{\partial h_{\ell,t}}$.¹⁵ These standard basis probes are tracer dyes through the opaque local transformation, one dye per channel. Figure 6 depicts this computation exactly.

In a single context $\mathbf{C}_{\mathcal{P}}$ with $S_{\mathcal{P}} = 100$,¹⁶ we can take an average of the $(100 + 99 + \dots + 1 =) 5050$ individual $\frac{\partial h_{L,t'}}{\partial h_{\ell,t}}$ matrices, each representing a $(h_{\ell,t}, h_{L,t'})$ pair from the context. The duality of these Jacobians is beautiful. For a single context, the 100 in the sum could be interpreted as the 100 final latents h_L that depend on $h_{\ell,0}$, or the 100 early latents $h_{\ell,0}, \dots, h_{\ell,99}$ informing the singular $h_{L,99}$. Figure 7 and Figure 8, respectively, show this exact duality.

Averaging over token position for a single context $\mathbf{C}_{\mathcal{P}}$ ’s latents gives us the mean influence of layer ℓ latents on future layer L latents.¹⁷

¹⁴It bears repeating: we’re not just saying that the *result* $R_{\ell}(h_{\ell,t})$ is dependent on $h_{\ell,t}$ —this is the nature of any function. We’re saying that the actual algorithm R_{ℓ} behaves differently for different activations.

¹⁵These e_d probes could be sent forward or backward equivalently—while we discuss and display the forward route, Anthropic’s released materials seeded backward passes. Either way, they traverse the frozen set of linear operators unlocked and saved in the actual full forward pass of the model and recover the exact same information.

¹⁶We fix a uniform context length ($S_{\mathcal{P}} = 100$ throughout) for concreteness. For a corpus of differing-length contexts, weigh each context’s contribution when forming J_{ℓ} so that longer ones do not dominate the mean.

¹⁷It should be made clear that while there are 5050 distinct relationships being averaged together in the above example, we only need d_{model} iterations to compute this mean for the whole context $\mathbf{C}_{\mathcal{P}}$. A forward pass seeded at layer ℓ propagates to *all* downstream positions in a single sweep, and the causal mask guarantees a source latent $h_{\ell,t}$ reaches only targets $t' \geq t$. So d_{model} forward or backward probes—one per model dimension—already carry every source’s influence to every target it can touch, filling the whole-context Jacobian without ever

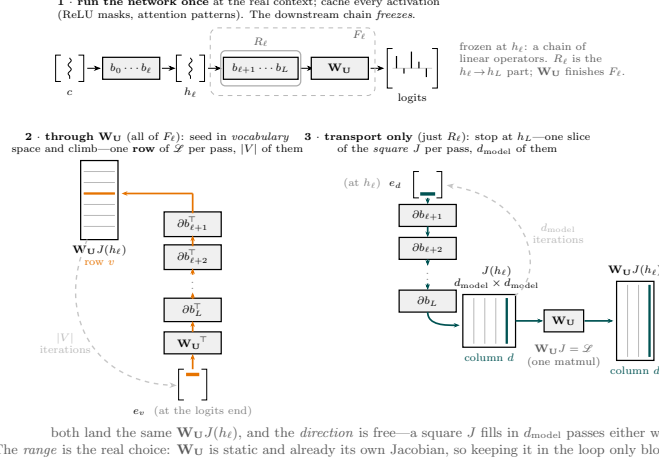
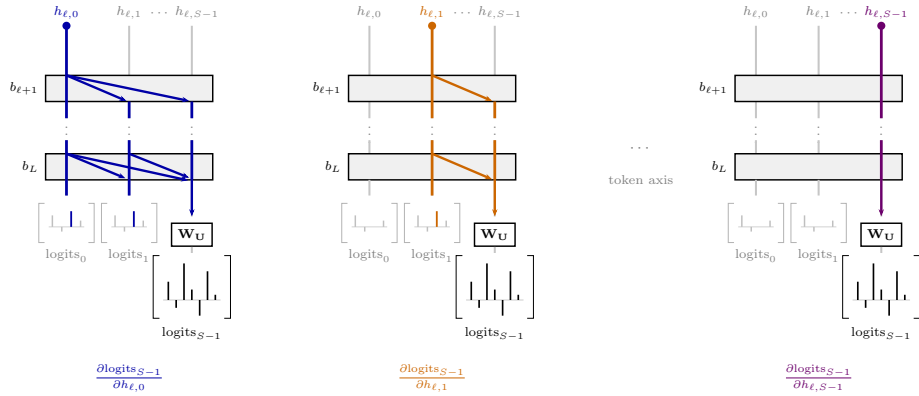


Figure 6: Two routes to the **J-lens** matrix $\mathbf{W}_U J(h_\ell)$, built here at one frozen context (before averaging into $\mathcal{L}_\ell$). One forward pass at the real context anchors everything, freezing the downstream chain into a stack of linear operators; both routes reuse that single cache. The choice that matters is **how far to differentiate**. **Through $\mathbf{W}_U$** (left) seeds a one-hot e_v in vocabulary space and climbs the transposes—including $\mathbf{W}_U^\top$ —depositing one row of $\mathcal{L}$ per pass, and so paying $|V|$ of them. **Transport only** (right) stops at h_L , filling the *square* transport Jacobian J ($d_{\text{model}} \times d_{\text{model}}$) one slice per pass and never touching $\mathbf{W}_U$; a single matmul $\mathbf{W}_U J$ then lifts the whole of it into $\mathcal{L}$. Direction is the lesser question: a square J costs d_{model} passes whether filled forward from h_ℓ or backward from h_L —it is drawn forward here only to show the column filling. Range is what separates them: $\mathbf{W}_U$ is a static matrix, already its own Jacobian, so carrying it inside the loop re-derives what we already have and inflates every pass into $\mathbb{R}^{|V|}$. For $(|V|, d_{\text{model}}) = (50\text{k}, 12\text{k})$, that is the $4\times$ discount.



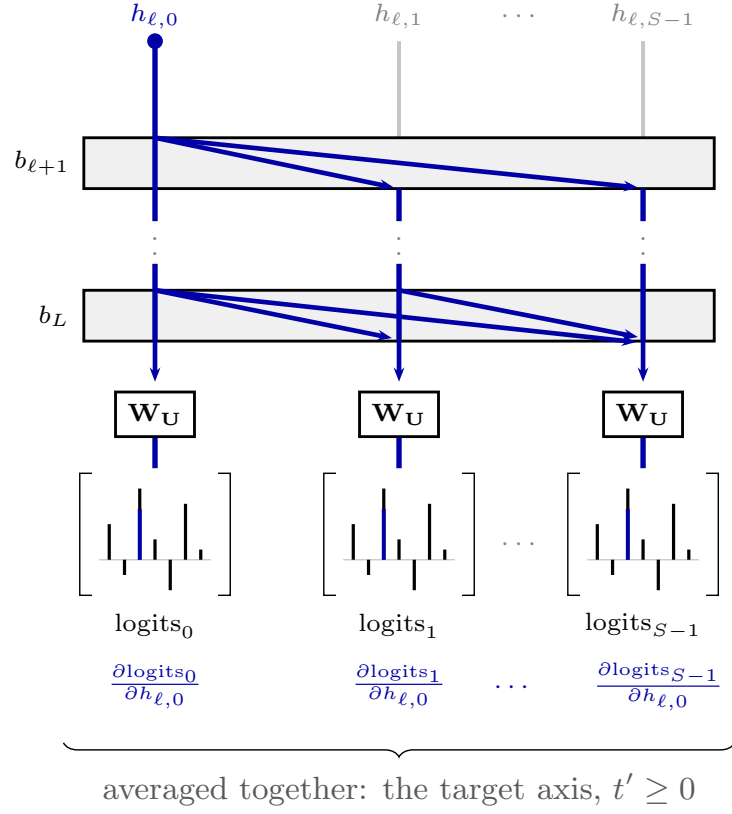


Figure 8: The complement to Figure 7. Here, we highlight not the activations affecting a single logit vector, but a single activation and the many logit vectors it influences. One latent, $h_{\ell,0}$, and every logit vector it touches—its causal *tails*.

$$J_\ell^\rho = \mathbb{E}_{t \leq t' < S_\rho} \left[\frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \right] = \mathbb{E}_{t \leq t' < S_\rho} \left[\frac{\partial R_\ell}{\partial h_\ell} \right]$$

Averaging $|\mathcal{P}|$ of these context-specific Jacobians J_ℓ^ρ gives us J_ℓ .

$$J_\ell = \mathbb{E}_{\rho \in \mathcal{P}} [J_\ell^\rho]$$

As we saw in the definition above, with J_ℓ , we have $\mathcal{L}_\ell$. Composition with $\mathbf{W}_\mathbf{U}$ is the static readout lens for layer L latents and J_ℓ already approximates the translation from layer ℓ . We’ve tied the directions of layer ℓ latents through layer L latents to tokens, at positions $t' \geq t$. This gives $\mathcal{L}$ bearing over future—not past, and not just *immediate* future—verbalization.¹⁸

3.2 Insightful Caveats

What is the epistemological cost of averaging the local Jacobians into a global lens? An individually computed $\frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ is exact, but indexed to a specific point in layer ℓ ’s activation space. The earlier ℓ is, the more of R_ℓ ’s higher-order curvature is being averaged away when we mean—first over positions into each context’s J_ℓ^ρ and then over $|\mathcal{P}|$ contexts—into the single J_ℓ . We average a layer’s map over all $(|\mathcal{P}| \times S)$ visited activations, pretending like local linear behavior is shared regionally when it really isn’t. Figure 9 attempts to stimulate intuitions about the information lost in local linear averaging of non-linear functions.

So then why average over contexts if we’re creating an increasingly approximated object by doing so? The goal of $\mathcal{L}$ is not to be an exact J of a specific logits vector, with respect to a specific h —it is manifestly to be an aggregated, generalized map. While the complexity of the field of local functions defined in F (by way of R)—the distinct attention paid to every unique context—is exactly what makes these models so powerful, the construction of $\mathcal{L}$ is a *pos-tulation* that there is *some* shared shape across the ways each context inspires speech in the model. Complex as F necessarily is, it is not some incompressibly arbitrary map. We lose information when we average local specificity into

visiting the 5050 pairs one at a time.

¹⁸Whether we fill this matrix by climbing backward or descending forward is the lesser question—the transport Jacobian J is square, so it costs d_{model} passes either way, and implementations generally climb, since backpropagation is the native idiom for it. The choice that earns its keep is *how far to differentiate*. We could carry the derivative all the way through $\mathbf{W}_\mathbf{U}$ into logit space, filling $\mathcal{L}$ a row at a time; but $\mathbf{W}_\mathbf{U}$ is a static matrix, and a static matrix *is* its own Jacobian—re-deriving it on every pass buys nothing, and seeding in vocabulary space inflates the loop from d_{model} to $|V|$. What these expected Jacobians really bring to the table is an approximation of the expected Jacobian of the network’s *nonlinear* processing, so that is the only stretch worth differentiating: from ℓ to L , $\mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{model}}}$, probing the nonlinearity and nothing else. Compose the readout once at the end. Figure 6 draws both routes and why they land in the same place.

regional (dare we say) global approximations, but what we retain has general, nontrivial causal insight (as the paper shows and we discuss in §??).¹⁹

Averaging is a hypothesis. Here is how we test it: we can measure the magnitude of difference ($\delta_{\ell,t,t'} = \left\| \frac{\partial h_{L,t'}}{\partial h_{\ell,t}} - J_\ell \right\|^2$) between each local Jacobian and the global J_ℓ , and then average this to get an overall expected **dispersion** measure ($\mathcal{D}_\ell = \mathbb{E}_{t' \geq t, \mathcal{P}}[\delta_{\ell,t,t'}]$)²⁰ for a given J_ℓ .²¹ $\mathcal{D}_\ell$ is proportional to (1) the higher-order curvature of R_ℓ and (2) variance in the sampled activations h_ℓ in the h -region sampled. These factors compound, schematically: $\mathcal{D}_\ell \propto \left\| \frac{\partial^2 R_\ell}{\partial h_\ell^2} \right\|^2 \cdot \mathbb{E} \|h_\ell - \bar{h}_\ell\|^2$.

$\mathcal{D}$ fills in the detail productively smeared by J and $\mathcal{L}$, complementing but not invalidating their causal importance.

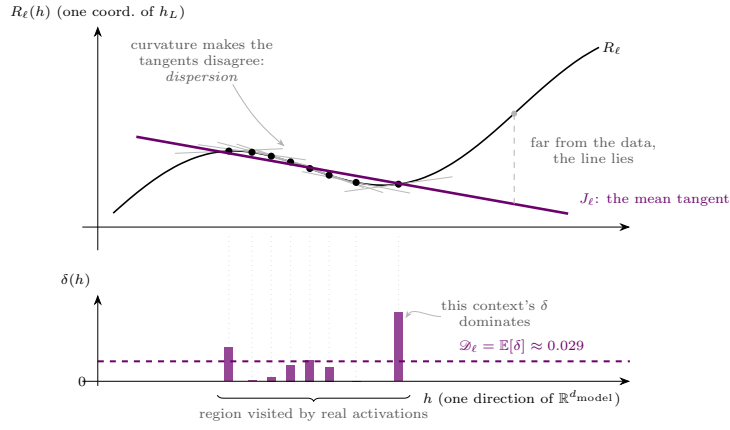


Figure 9: What the lens is: the average of the local linear behaviors $J_\ell(h) = \frac{\partial h_L}{\partial h}$ —i.e. the atom $\frac{\partial h_{L,t'}}{\partial h_{\ell,t}}$ at each visited latent $h = h_{\ell,t}$ —across the activations the model actually visits (dots; one gray tangent each). Where R_ℓ curves within the visited region, the tangents fan—that spread is the dispersion $\mathcal{D}_\ell = \mathbb{E} \|J_\ell(h) - J_\ell\|^2$. And the mean tangent is a local instrument: far from the sampled region, the line and the function part ways. The lower panel makes the spread concrete: each bar is one visited latent’s $\delta = \|J_\ell(h) - J_\ell\|^2$ —how far its tangent tilts from the mean—and $\mathcal{D}_\ell$ is simply their average, pulled up here by the lone high-curvature latent at the region’s right edge.

¹⁹For what it is worth, *training* adheres to the same postulation with respect to the effect of averaging over nonlinearity. It is hard to deny its success in postulating so. $\mathcal{L}$ takes the same kinds of means and shows that—in the trained model, in *activation* space—specificity is not the entire story.

²⁰By the law of total variance the total splits as $\mathcal{D}_\ell = \mathcal{D}_\ell^w + \mathcal{D}_\ell^b$: a *within-context* part $\mathcal{D}_\ell^w = \mathbb{E}_\rho \mathbb{E}_{t' \geq t} \left\| \frac{\partial h_{L,t'}}{\partial h_{\ell,t}} - J_\ell \right\|^2$ (atoms scattering around their own prompt’s mean—local curvature) plus a *between-context* part $\mathcal{D}_\ell^b = \mathbb{E}_\rho \|J_\ell^p - J_\ell\|^2$ (prompt-means disagreeing). We headline the total, but the between-part is the sharper object—it alone asks whether averaging *over contexts* was warranted, i.e. how well the lens generalizes across them. The identity is exact provided both stages share one weighting over contexts: automatic for uniform S_ρ , and for mixed lengths one weights each context by its (t, t') pair-count.

²¹Since the readout $\mathbf{W}_\mathbf{U}$ is a shared constant, it drops out of the difference, and only the nonlinear transport actually varies.

4 J-space

Where $\mathcal{L}$ shines in the *synthesis* it makes of causal relationships into a single object, it blinds with its potent density. The **J-space** is a kind of prism we fit to the information that pours into $\mathcal{L}$, reducing it to a sparse few of its directions and focusing its information into distinct beams of practical insight. $\mathcal{L}$ colors the d_{model} -sized latent states of the model with what it means for all $|V|$ concepts in its available grasp, but do we feel intuitively that every thought in our heads should be put in relation to *every* word we can possibly utter? Any h must be reducible to only those directions in token space that really count, right? Beyond the reduction the **J-space** provides for the purposes of interpreting internal state, if we want to affect change in the model’s behavior, we’d hope that we wouldn’t have to tweak *every single* source, but only a privileged few, right?

Knowing the normalized alignment vector $\mathcal{L}(h)$ of any latent is valuable, but $\mathcal{L}$ itself (and the vocabulary it speaks to!) is highly redundant. If h scores highly with a token, “dog”, then it will likely align with tokens such as “dogs”, “canine”, and “puppy”.²² Knowing one of these practically tells us the other. Taking the top 10 scoring tokens from $\mathcal{L}(h)$ might only give us 2 or 3 truly distinct concepts.

The **J-space** $\mathcal{S}$ is defined to bring information-retaining **sparsity** to the redundancy of $\mathcal{L}$ -space. $\mathcal{S}$ is an *intensionally* as opposed to *extensionally* defined space, never fully enumerated as an object like $\mathcal{L}$ but instead specified by a constrained recipe for finding its members.²³ The **J-space decomposition** $\mathcal{S}(h)$ of an activation h is the closest point *in the J-space* to h itself. Once $\mathcal{L}$ -space is made sparse, the meaningful conceptual decomposition of the residual stream is fast-tracked and principled, allowing for efficiently pointed interventions.

4.1 Computation

The **J-space** $\mathcal{S}_\ell$ in layer ℓ is defined as the set of all points reachable by a sparse, nonnegative decomposition of up to k rows of $\mathcal{L}$, where k is our sparsity constraint. Anthropic denotes it as a *union of cones* in $\mathbb{R}^{d_{\text{model}}}$, where each cone is made up of a set of k directions (rows drawn from $\mathcal{L}$), each with a nonnegative coefficient. Writing m_v for the row of $\mathcal{L}$ belonging to token v , and collecting the coefficients into a single sparse, nonnegative vector $c \in \mathbb{R}_{\geq 0}^{|V|}$ with at most k nonzero entries ($\|c\|_0 \leq k$), the space is

$$\mathcal{S} = \{ \mathcal{L}^\top c : c \geq 0, \|c\|_0 \leq k \}, \quad \mathcal{L}^\top c = \sum_{v=1}^{|V|} c_v m_v$$

The **J-space decomposition** $\mathcal{S}(h)$ of a latent h is then defined as the point in $\mathcal{S}$ that is closest to h .

²²These words specifically may not be tokens in real LLMs, but it is an idealized example.

²³For one thing, it is an *uncountably* infinite space—a union of finitely many cones, but each cone a continuum of points—so we could never compact it into an object like $\mathcal{L}$.

$$\mathcal{S}(h) = \mathcal{Z}^\top c^*, \quad c^* = \arg \min_{\substack{c \in \mathbb{R}_{\geq 0}^{|V|} \\ \|c\|_0 \leq k}} \|\mathcal{Z}^\top c - h\|_2^2$$

c^* is our prism on the overcomplete white light that is $\mathcal{Z}$. Once fit, c^* sifts the k directions in $\mathcal{Z}$ -space ($\in \mathbb{R}^{d_{\text{model}}}$) that best reconstruct h . If each row of $\mathcal{Z}$ denotes a direction of steepest logit-increase for each token, then $\mathcal{S}(h)$ puts together a non-redundant report of the k directions in token space that best describe the model’s disposition.

While the notation defining $\mathcal{S}(h)$ is succinct, choosing the best k rows out of a $|V|$ -term dictionary is NP-hard, so we solve it greedily. One objective—*reconstruct h as closely as possible with only k rows*—forces two oscillating, complementary operations: *select* directions against the residual r (which progressively shrinks during the optimization), but always *fit* against h . Every *next* direction m is the one most aligned with *what still needs to be covered* (the residual, r). After each next direction selection, we see how well the working set of directions reconstructs the ultimate objective, h . *Diversity* among the k rows of $\mathcal{Z}$ that we select is never an imposed rule—it falls out of greedily *selecting* against the residual but *fitting* against the target. The selected rows are not thereby *mutually orthogonal* (with an overcomplete, correlated dictionary they generally are not); what emerges is only that we never waste one of our k slots re-selecting a direction the working set already covers, since after each refit the residual is (near-)orthogonal to everything we have already captured. Algorithm 1 displays the *outer* algorithm for selecting directions, while Algorithm 2 (FIT) specifies how those directions are fit. Because FIT re-solves *all* of the active coefficients against h at every round—rather than freezing past coefficients and only appending the new one—the procedure is, strictly, nonnegative *orthogonal* matching pursuit, not the original matching pursuit.²⁴

Algorithm 2 should seem familiar: it is gradient descent! Write the error as $e \equiv \hat{s} - h = \sum_w c_w m_w - h = -r$, where $\hat{s}$ is the running reconstruction of h under the current coefficients, so the loss is

$$L(c) = \|\hat{s} - h\|_2^2 = \|e\|^2 = e \cdot e$$

Differentiating with respect to a single coefficient c_v ,

$$\frac{\partial L}{\partial c_v} = \frac{\partial}{\partial c_v} (e \cdot e) = 2e \cdot \frac{\partial e}{\partial c_v},$$

the inner derivative picks out a single term,

$$\frac{\partial e}{\partial c_v} = \frac{\partial}{\partial c_v} \left(\sum_w c_w m_w - h \right) = m_v$$

²⁴Orthogonal matching pursuit: Y. C. Pati, R. Rezaifar, and P. S. Krishnaprasad, *Orthogonal Matching Pursuit: Recursive Function Approximation with Applications to Wavelet Decomposition*, Proc. 27th Asilomar Conf. on Signals, Systems and Computers, 1993.

Algorithm 1 $\mathcal{S}(h)$

Require: latent $h \in \mathbb{R}^{d_{\text{model}}}$; J-lens rows $\mathbf{m}_v$ for $v = 0, \dots, |V| - 1$; sparsity budget k

Ensure: sparse nonnegative $c \in \mathbb{R}_{\geq 0}^{|V|}$ with $\|c\|_0 \leq k$; decomposition $\mathcal{S}(h) = \mathcal{L}^\top c$

- 1: $r \leftarrow h$; $A \leftarrow \emptyset$; $c \leftarrow \mathbf{0}$ $\triangleright$ residual, active support, coefficients
 - 2: **for** $j = 1$ **to** k **do** $\triangleright$ outer loop: grow the support
 - 3: $v^* \leftarrow \arg \max_v \frac{\langle \mathbf{m}_v, r \rangle}{\|\mathbf{m}_v\|}$ $\triangleright$ select against the residual r — the fastest error drop
 - 4: $A \leftarrow A \cup \{v^*\}$ $\triangleright$ selecting against r , not h , keeps the support diverse
 - 5: $c_A \leftarrow \text{FIT}(A, h)$ $\triangleright$ fit the weights against h itself (Alg. 2)
 - 6: $r \leftarrow h - \sum_{v \in A} (c_A)_v \mathbf{m}_v$ $\triangleright$ what $\mathcal{S}$ still cannot explain
 - 7: **end for**
 - 8: **return** c (zero off A), $\mathcal{S}(h) = \mathcal{L}^\top c$
-

Algorithm 2 $\text{FIT}(A, h)$

Require: active support A ; latent h ; J-lens rows $\mathbf{m}_v$; step size η

Ensure: $c_A \geq 0$ minimizing $\|\sum_{v \in A} (c_A)_v \mathbf{m}_v - h\|_2^2$

- 1: $c_A \leftarrow \mathbf{0}$
 - 2: **repeat**
 - 3: $r \leftarrow h - \sum_{v \in A} (c_A)_v \mathbf{m}_v$ $\triangleright$ residual under the current weights
 - 4: $g_v \leftarrow -\langle \mathbf{m}_v, r \rangle \quad \forall v \in A$ $\triangleright$ gradient: (minus) each row's alignment with r
 - 5: $c_A \leftarrow \max(\mathbf{0}, c_A - \eta g)$ $\triangleright$ descend, then project back onto $c_A \geq 0$
 - 6: **until** $\|r\|$ stops shrinking
 - 7: **return** c_A
-

since h and every c_w with $w \neq v$ are constant in c_v . Substituting back,

$$\frac{\partial L}{\partial c_v} = 2e \cdot m_v = 2(\hat{s} - h) \cdot m_v = 2(-r) \cdot m_v = -2\langle m_v, r \rangle$$

At every turn of the optimization, r is the vector representing all of h that we have not reconstructed. It is a *direction* in which we want to move, not a point at which we want to arrive. The more we nudge $\sum_{v \in A} (c_A)_v m_v$ towards r , the more r disappears.²⁵ We want to adjust each coefficient c_v so it supports this directional goal and brings its corresponding m_v to where it needs to be to cancel more of r . What is the relationship between c_v and r ? It is exactly (proportional to) $\langle m_v, r \rangle$!

The motivation for the **J-space** decomposition $\mathcal{S}(h)$ of a latent h is plain: positively include the k distinct token-influence directions of h -space that can reconstruct h the best. “Describe the model’s disposition in k words, each with a weight.” The *algorithm* for computing it, though, is even more plain and elegant. At every turn of the outer algorithm, select the direction that best eats up the work we have left to do (r).²⁶ Then for each working set of directions, figure out the best way to combine them to reconstruct the information we actually want to reconstruct, h . The algorithm is so nice because its goal is so interpretable and intuitive and its methods are so familiar to anyone who has worked with neural networks. Interestingly, it was originally introduced (under the name of *matching pursuit*) in the context of continuous signal processing by Mallat and Zhang (1993).²⁷ Continuous sinusoidal waves that are composed of pure frequencies, each at a specific amplitude (and phase), resemble vectors that are composed of different directions. The motivation to deconstruct them is the same: *which frequencies are present in this signal?* Better yet, *what are the **top k** frequencies that could approximate this signal?* Each pure frequency brings something distinct to the table, like each direction of a vector space. (1) How do you find the single pure frequency that best aligns with a sinusoid (be it a raw signal or a residual)? Dot product! This gives us the Discrete Fourier Transform. (2) If you have a fixed set of pure frequencies, how do you figure out their best combination to reconstruct a sinusoid? Gradient descent! Figure 10 celebrates this alignment in methods and motivations, honoring the original instantiation of the matching pursuit algorithm which we use to compute $\mathcal{S}(h)$, and stimulating your intuition for what reconstruction really looks like.

4.2 Insightful Caveats

So we can reconstruct any latent h with a selection of a few key disposition vectors...what about the rest of the directions tied up in h ? In the same vein of

²⁵Because $r = h - \sum_{v \in A} (c_A)_v m_v$!

²⁶This is what buys us the diversity noted above. In an idealized view, whatever directions are grabbed in one round against r are ‘claimed’ from then on, removed from r' , and ‘unwanted’ by the subsequent dot products against r' .

²⁷S. G. Mallat and Z. Zhang, *Matching Pursuits with Time-Frequency Dictionaries*, IEEE Trans. Signal Processing, 41(12):3397–3415, 1993.

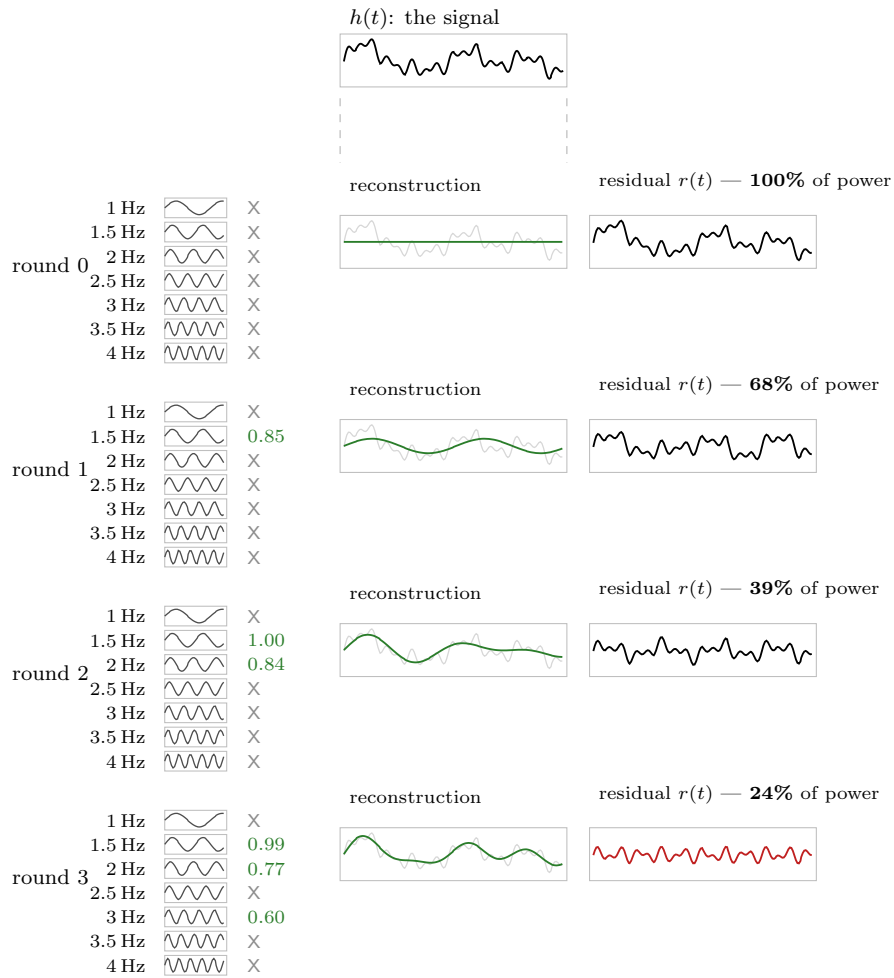


Figure 10: Matching pursuit on a continuous signal, round by round: each round selects the pure frequency (left) most aligned with the current residual, refits the active coefficients to reconstruct the target signal $h(t)$ (center; the true signal is ghosted behind the running reconstruction), and shrinks the residual (right). Percentages are the residual's share of the signal's own power about its mean level, $\|r\|^2/\|h\|^2$ —that is, $1 - R^2$ for that round's fit. The same select–fit–shrink loop computes $\mathcal{S}(h)$.

asking what we lose when we average local Jacobians to get J and $\mathcal{L}$, we can ask about the epistemological consequences of very shrewdly dismissing *most* of our $\mathcal{L}$ vectors to get $\mathcal{S}$ and $\mathcal{S}(h)$. What information are we losing? (...What information did we care about reconstructing?)

A first thought might be: we only keep k directions to reconstruct all of h , so we only keep $\frac{k}{d_{\text{model}}}$ of the information in h . This would only be true if the rows of $\mathcal{L}$ (from which we draw our k) were orthogonal and linearly independent. This is manifestly not the case.²⁸ Trained neural networks do not spread information uniformly through the spaces they’re afforded—they are not static tables. Training is a process of *compressing*, *abstracting*, and *discovering* useful redundancies in the representation and apprehension of information. Because of this, the causal tendrils flowing from any given node within the model are braided and entangled, giving the model the power to make high-leverage decisions, but also blinding the path backwards through those decisions. $\mathcal{L}$, as we know, was explicitly constructed by pulling derivatives backward, trying to reverse and untangle the causal course that determined the behavior we observed.

So if $\mathcal{L}$ is highly redundant, then maybe k directions really can reconstruct almost all of the information in a given h ...can it? At this point, we have to ask ourselves what we mean by *information*. Information content is always in the eye of the beholder—Shannon’s information depends on *expectation*.²⁹ In what ways can we measure the information in latents h and compare it with that which is reconstructed in $\mathcal{S}(h)$?

One neutral measure is simply the statistical variance. Of all the latents sampled in the process of computing $\mathcal{L}$, what was their average information as denoted by variance? If they were all the same, variance would be null and therefore we could see that the information each one offers up on its own would be null as well. Loosely, then: the spread of a layer’s sampled activations about their average. This spread is a proxy for informational distinctness across the sampled activations of a layer—which, we’d hope, are distinct and decisive. How much of that spread, then, do $\mathcal{S}$ decompositions cover? The natural quantity has already been put to good use—the *residual*, our standing measure of what is left *uncovered*.

So what did Anthropic find? Despite the success of $\mathcal{S}$ —to be discussed in later sections—the variance covered was typically only as high as 10%.³⁰ In

²⁸For one thing, there are (typically) more rows ($|V|$) in $\mathcal{L}$ than dimensions in each row d_{model} . So even if every row could only be a basis vector, some would be duplicates, hence killing orthogonality.

²⁹To be clear, though, we are *not* talking about Shannon information here. It could be fit to the scenario, but we’re sticking with more primitive measures, and are using the term in a general, intuitive fashion.

³⁰The paper reports the figure without giving a formula: the $\mathcal{S}$ component “typically accounts for only a small fraction of total activation variance (varying by layer, but never more than 10%),” and, for concept vectors specifically, “carries a median of only 6–7% of the concept vector’s variance.” The released code covers the lens itself, not this decomposition, so the precise quantity is not recoverable—which is reason enough to hold the number loosely and not to lean on its third digit.

typical LLMs, this is far higher than $\frac{k}{d_{\text{model}}}$, but it is still a rather lower number than we might expect. The bulk of the variance of activations is unseen by the $\mathcal{S}$ decomposition...why?

We said that information is in the eye of the beholder. What was the gaze that guided our constructed $\mathcal{L}$ —the source from which we drew $\mathcal{S}$? How does it *see* the raw data stored in a single latent? This is the crux: the computation of the $\mathcal{L}$ only ever saw latents in terms of their downstream causal impact on logits. The power of $\mathcal{L}$ is that it is *not* a representation of static state h , but instead disposition, informed by derivatives blaming h . $\mathcal{S}(h)$ doesn’t reconstruct h through some geometrically neutral gaze—like that of raw variance—but instead does so in the basis of causal leverage itself. These top k directions picked from such a basis, yes, leave lots of variance on the table, but every one of them is, by construction, a direction of causal bearing on the network’s outputs. Furthermore, the directions left out of $\mathcal{S}(h)$, while representing the vast majority of h ’s variance, hold almost no causal weight on what is actually said.

With this in mind, we see $\mathcal{S}$ for what it is—the causal, dispositional profile of h —and what it isn’t—a naive, generalist reconstruction of the static information frozen into the activations of any layer. The results explored in the following section complement the above discussed caveat, telling us that sheer *volume* and true *influence* can be separated (something we already know, for example, in DNA). What *makes J-vision* is the shedding of noise for causal signal, collapsing our focus to a crystallized, privileged subset—just as training did.

5 J-vision In Language Models

The **J-lens** and **J-space**—heralding a philosophy and family of techniques I think of as **J-vision**—are principled mathematical objects that demystify causal privilege within LLMs. Their definitions and construction are sensible and self-evidently answer good questions about LLMs. They allow us to *read* the states of the model as it computes, and put their complexity into legible context. Anthropic showed several fascinating examples of **J-monitoring** the latents in models as they performed multi-step reasoning, seeing sensible intermediate or supporting concepts present themselves along the way. For example, when prompted with “**calc:** (4+17)*2+7 =”, the $\mathcal{L}$ readout at $h_{\dots, “=”}$ showed “21”, “42”, and “49”, respectively, at ever deeper layers. Similarly, when prompted “**The color of the planet fourth from the sun is**”, the readout at “**is**” showed “color”, “Mars”, and eventually “red” at ever deeper layers. These examples (and others, beautifully displayed in the paper) show that $\mathcal{L}$ finds the model’s intermediate reasoning in the course of determining what tokens to sample. This alone is incredibly useful and interesting, but beyond just reading the model’s reasoning, we can intervene on it.

5.1 J-intervention

In this section, we explain the methods and variants of two kinds of interventions: *swaps* and *injections*. In all cases, these interventions take the form of $h_{\text{intervened}} = h + \Delta h$. We are always adding a vector $\Delta h \in \mathbb{R}^{d_{\text{model}}}$. Further, wherever variants of an intervention are compared against one another, we want to make sure Δh has a standardized magnitude. The whole point of these interventions is for them to be mutually- or null-comparable, and holding magnitude standard allows us to avoid achieving some distinct effect merely by injecting ‘unnatural’ magnitude. In keeping with the vectors-as-arrows, differences-as-angles paradigm we’ve discussed as central to transformers, we want our interventions to look more like rotations.

There are two key techniques resident in the swaps and injections that we want to make sure are crystal clear. The first is **orthogonal projection** of a latent h onto a subset of directions $\in \mathbb{R}^{d_{\text{model}}}$. (The second technique, **concept vectors**, will be explained in the next section.) The $\mathcal{S}$ decomposition of a latent seeks a kind of projection onto k directions from $\mathcal{L}$, but it is a constrained projection. The coefficients assigned in the $\mathcal{S}$ basis must be nonnegative, by our construction, for several reasons discussed above—such as seeking only *positive* inclusion of concepts-denoted-by-tokens, avoiding the confusion of what *negative* inclusion means. The $\mathcal{S}$ decomposition is meant to be interpretable; the projections we want to make for swaps have different goals. For swaps, we essentially want to look at a subset of directions $\in \mathbb{R}^{d_{\text{model}}}$, project an activation h onto their span, and exchange their respective coefficients. The swap should be an involution meaning that applying the operation twice leaves h unchanged. For the most basic swap of two directions, we’ll choose a basis $\mathcal{V} \in \mathbb{R}^{d_{\text{model}} \times 2}$, with vector components $\{v_0, v_1\}$, each $\in \mathbb{R}^{d_{\text{model}}}$. We’ll then seek a coefficient vector $\mathcal{C} \in \mathbb{R}^2$ with scalar components $\{c_0, c_1\}$, all of whom seeking to minimize $\|h - \mathcal{V}\mathcal{C}\|^2$. With no nonnegativity constraints on $\mathcal{C}$, our objective is:

$$\nabla_{\mathcal{C}} \|h - \mathcal{V}\mathcal{C}\|^2 = -2\mathcal{V}^\top (h - \mathcal{V}\mathcal{C}).$$

Setting this gradient to 0, we get:

$$\begin{aligned} -2\mathcal{V}^\top (h - \mathcal{V}\mathcal{C}) &= 0 \\ \mathcal{V}^\top (h - \mathcal{V}\mathcal{C}) &= 0 \end{aligned}$$

Thus we see that if we can minimize that objective, we’ll get a projection that is orthogonal to the residual it leaves behind—i.e. we’ll be able to work in this projected space without touching the rest of the activation $h - \mathcal{V}\mathcal{C}$. No constraints on this optimization also means we have a closed form solution to $\mathcal{C}$:

$$\begin{aligned} \mathcal{V}^\top (h - \mathcal{V}\mathcal{C}) &= 0 \\ \mathcal{V}^\top \mathcal{V}\mathcal{C} &= \mathcal{V}^\top h \\ \mathcal{C} &= (\mathcal{V}^\top \mathcal{V})^{-1} \mathcal{V}^\top h \end{aligned}$$

This term applied to h on the right is known as the Moore-Penrose Pseudoinverse, which we denote with a dagger:

$$\mathcal{V}^\dagger = (\mathcal{V}^\top \mathcal{V})^{-1} \mathcal{V}^\top.$$

As can be seen in the above equations, $\mathcal{V}^\dagger$ is a readout filter on h that gives us exactly the coefficients of the point in $\mathcal{V}$'s span closest to h . How does it work? By first multiplying $\mathcal{V}^\top$ by h , we get dot products of each $v_i \in \mathcal{V}$ with h , values proportional to each direction's alignment with h . This is a useful starting point for the optimized projection we want, but as coefficients for reconstruction, they over-count any overlap that exists among the directions in $\mathcal{V}$. If v_i and v_j are not orthogonal, an amount proportional to $\langle v_i, v_j \rangle$ is inflating the i -th and j -th indices of our would-be coefficient vector $\mathcal{V}^\top h$.

What we want instead is simple to state: each direction in $\mathcal{V}$ should carry exactly the weight it *distinctly* contributes to h . That is precisely what the rows of $\mathcal{V}^\dagger$ compute:

$$c_i = \frac{\langle v_i^\perp, h \rangle}{\|v_i^\perp\|^2},$$

where $v_i^\perp$ is v_i with every *other* direction of $\mathcal{V}$ projected out.

In coordinates, that correction is carried by the inverse Gram matrix, $G^{-1} = (\mathcal{V}^\top \mathcal{V})^{-1}$, whose i -th row supplies a correction coefficient for each term $(\mathcal{V}^\top h)_j$. On G^{-1} 's diagonal (G_{ii}^{-1}) sits $1/\|v_i^\perp\|^2$, normalizing by however much of v_i no other direction could account for. Off the diagonal ($G_{ij}^{-1}, j \neq i$) the entries are *negative*, with magnitude growing in $\langle v_i, v_j \rangle$: the more direction i aligns with direction j , the more of j that i 's readout $(\mathcal{V}^\top h)_i$ drew in, and the more has to be subtracted back out. $\mathcal{V}\mathcal{C} = \mathcal{V}(\mathcal{V}^\dagger h)$ then gives us the closest point in the span of $\mathcal{V}$ to h , the residual $(h - \mathcal{V}\mathcal{C})$ being totally orthogonal to the span (by construction).

Once we have this orthogonal projection onto two directions, we can cleanly swap them. Defining $\sigma\{c_0, c_1\} = \{c_1, c_0\}$, then:

$$h_{swapped} = h + \mathcal{V}(\sigma\mathcal{C} - \mathcal{C})$$

$$h_{swapped} = h + \mathcal{V}(\{c_1 - c_0, c_0 - c_1\}).$$

In an isolated span, we simply switch the roles played by one direction with that of another. When this second term is added back to h , the direction v_0 is left with only c_1 scale, because of the cancelling in the $c_0 v_0 + c_1 v_0 - c_0 v_0$ term that arises from this summation. Swaps are clean, isolated involutions within a strict span.

Now the question becomes what directions would be interesting to put into $\mathcal{V}$? Here is an example of a basic swap Anthropic employed. The model was prompted:

Human: Think of a sport. Answer in one word.
Assistant:

In $\mathcal{L}(h_{92}, \cdot)$, “Soccer” and “Basketball” were the two highest scoring tokens.³¹ They then performed a swap, where $\mathcal{V} = \{m_{\text{“Soccer”}}, m_{\text{“Rugby”}}\}$. After the swap, $\mathcal{L}(h_{92}, \cdot)$ ranked “Rugby” highest and, duly, “Rugby” had the largest downstream logit and therefore the highest odds of verbalization. They constructed many similar swaps, the statistical efficacy of which will be explained in the next section.

Next, we have concept injections. Rather than swapping two concepts in an orthogonal subspace, here we simply add a direction related to a concept. They give the model the following prompt:

Human: ...[*explanation that a thought may be injected*]...
Assistant: Understood.
Human: Do you detect an injected thought? If so, what is the thought about?
Assistant: Yes, I detect an injected thought. The thought is about the word "

In one example, when they submit this prompt, one of the top tokens in the $\mathcal{L}$ readouts across the context is “elephant”. They then add (a rescaled copy of) the $\mathcal{L}$ row for “lightning” wherever “elephant” was present, and the model reliably amplifies “lightning” in those $\mathcal{L}(h)$ readouts as well as in its ultimate verbal report. This shows that injecting based on the $\mathcal{L}$ can steer behavior; it doesn’t yet show that the $\mathcal{S}$, though, is a privileged seat for steering. This brings us to two similar experiments that seek to show the $\mathcal{S}$ as a *privileged* seat for intervention...

5.2 Clamped Concept Swaps In $\mathcal{S}$

The two following experiments depend on the notion of “concept vectors”. They compute concept vectors by recording the residual activations for prompts of the format

Human: Tell me about {*concept*}

mean-subtracted over a baseline set of 100 other concepts. In other words, they isolate the degree to which any concept’s activation (in a prompt in the above format) differs from the average activations in 100 prompts of this sort. These concept vectors are $\in \mathbb{R}^{d_{\text{model}}}$, and can be $\mathcal{S}$ -decomposed! They use the same decomposition method explained in §4.1—here with $k = 16$ —and keep both the **J-space** component $\mathcal{S}_{\text{concept}} = \mathcal{S}(h_{\text{concept}})$ and the non-**J-space** component $\neg\mathcal{S}_{\text{concept}} = h_{\text{concept}} - \mathcal{S}_{\text{concept}}$. They find that the **J-space** component across concepts and layers holds only a median of 6–7% of concept vector variance, with the remaining variance residing outside of the $\mathcal{S}$.

³¹In fact, they showed that “Soccer” and “Basketball”, and “soccer” and “basketball” claimed the four highest scores.

With these bifurcations of any $h_{concept}$, we have new vectors $\in \mathbb{R}^{d_{model}}$ that can be used for perturbations and compared for their steering efficacy. They first repeat the swap experiments with three variants (denoted under the same “Soccer”-“Rugby” example, to be concrete):

1. Swap $\mathcal{S}_{concept: \text{“Rugby”}}$ for $\mathcal{S}_{concept: \text{“Soccer”}}$ instead of $\mathcal{L}$ rows for either token
2. Swap $\neg \mathcal{S}_{concept: \text{“Rugby”}}$ for $\neg \mathcal{S}_{concept: \text{“Soccer”}}$ instead of $\mathcal{L}$ rows for either token
3. Swap $\neg \mathcal{S}_{concept: \text{“Rugby”}}$ for $\neg \mathcal{S}_{concept: \text{“Soccer”}}$, but clamp $\mathcal{S}_{concept: \text{“Soccer”}}$ coefficients to those in the clean (“Soccer”-favoring) forward pass in all layers

Variants 1 and 2 are straightforward: the original swap had $\mathcal{V}$ with each row being a $\mathcal{L}$ row, but now those are concept activations’ $\mathcal{S}$ components (or non-components) of the same size. Whereas the original swapping experiment—with $\mathcal{L}$ rows—saw 88% success, variant 1 was still able to achieve 59% success, while variant 2 only succeeded 5% of the time. Although it holds 93% of concept activation variance compared with the 6–7% held in $\mathcal{S}$ decompositions, the non-**J-space**-component of a concept activation has almost no causal weight in swap interventions.

Variant 3 seeks to isolate the $\neg \mathcal{S}$ swap to strictly the activation regions outside of the $\mathcal{S}$ in any layer. Instead of just swapping, variant 3 swaps and then performs a clamp operation to keep the swapped-out concept’s $\mathcal{S}$ component intact. How does it do such a surgical clamp? Using the same approach as the swap itself—which isolates the span of two directions and exchanges their coefficients—we isolate the span of $\mathcal{V}_{concept: \text{“Soccer”}}$ (which has $k = 16$ rows that are the $\mathcal{L}$ rows that make up $\mathcal{S}_{concept: \text{“Soccer”}}$), find $c_{clean} = \mathcal{V}_{concept: \text{“Soccer”}}^\dagger h_{clean}$, and obtain

$$h_{clamped} = h_{swapped} + \mathcal{V}_{concept: \text{“Soccer”}} (c_{clean} - \mathcal{V}_{concept: \text{“Soccer”}}^\dagger h_{swapped}).$$

All in all, it means we freely swap activations within $\mathcal{V}_{concept: \text{“Soccer”}}^\perp$ —everything orthogonal to those clamped rows—but do not let the span of $\mathcal{V}_{concept: \text{“Soccer”}}$ ever change, even downstream.³² Variant 3’s swap success rate falls to *zero*! This is incredible evidence that the $\mathcal{S}$ is a privileged causal region for verbalization, shown even stronger in further three-pronged experiments.

This three-pronged followup is also performed on the injection experiments from above, using the same clamping mechanism. For injections, the success with pure $\mathcal{L}$ -row injections reaches $\sim 90\%$, variant 1 achieves $\sim 80\%$, variant 2 $\sim 10\%$, and variant 3 falls to “near 0%”. This suggests essentially the same implications for verbal report as did the swap experiments above.

³²To be clear, the formulae in this section are not made explicit in Anthropic’s paper or code, but we’re inferring the mechanism likely used for these operations under their stated objectives and constraints.

The most staggering three-pronged results come from their multi-hop reasoning swaps. For example, they prompt the model with “**Fact: The number of legs on the animal that spins webs is**”. Unbothered, the model reliably gives the highest logit to “8”. Swapping the $\mathcal{L}$ row for “**ant**” with that of “**spider**” sees “6” rise to the highest logit $\sim 60\%$ of the time. They then recompute concept vectors under a similar method to that described earlier, but adjusted for “multi-hop representations” of concepts, and repeat the three $\mathcal{S}$ -related variants of the swaps. Variant 1 sees success hold at 61%—in fact edging out the raw $\mathcal{L}$ swap—with variant 2 now working 28% of the time. While still half as successful as the $\mathcal{S}$ swaps in variant 1, this $\neg\mathcal{S}$ swap still works nontrivially well. Why? The Anthropic team hypothesized that “**ant**” may have been re-routed into the $\mathcal{S}$ at later layers of the model after the earlier swaps—and variant 3 is the perfect experiment to test that hypothesis. When the concepts’ $\mathcal{S}$ directions were clamped to the clean-pass coefficients after the $\neg\mathcal{S}$ swaps, success fell to 6%!³³ While even before the clamps the verbal report experiments showed strong causal preference to the $\mathcal{S}$, this multi-hop experiment makes it decisively clear that if you disallow perturbations to affect the $\mathcal{S}$, you will not succeed in steering the model’s behavior.

5.3 Further J-vision

Anthropic’s paper showcases many more readings and interventions sanctioned by the $\mathcal{L}$ and $\mathcal{S}$ and, even if you skip the mechanisms, the visualizations (some of which are interactive) are an exciting delight. Additionally, they have collaborated with **Neuronpedia** to give the public extensive access to **J-vision** in LLMs on an interactive website. It is the most profound way to familiarize yourself with these tools short of replicating them yourself.

Finally, there are many questions that we can imagine asking with **J-glasses** affixed to our eyes. The class of research I’d most like to see or conduct involves measuring *dispositional abstraction* in terms of $\mathcal{L}$ and $\mathcal{S}$. Abstraction is a—if not *the*—foundational faculty in intelligent systems. It takes many forms and could be defined in many ways. Neural networks, at a basic level, are constructed to embody abstraction in the way they conduct sequential *templatic* processing. Any row of neurons—specified by a matrix which acts on an incoming vector—is just a series of templates which score the incoming vector for its fit in each template. The point of having dot products, wrapped in nonlinear activation functions is so that many specifically-different inputs can produce the same activations within a model—i.e. the model can *see* them as the same thing. This is abstraction: seeing the similar in two things that are, on their surface, different.

As I suggest, activation vectors are meant to be abstract representations of the input (ever more so as you go deeper in the model’s layers). But in LLMs, activations are only a means to an eventually verbalized end. The $\mathcal{L}$ is exactly

³³These probabilities refer to experiments of the same form as the “**spider**”-“**ant**” version made concrete here, but with different multi-hop facts used.

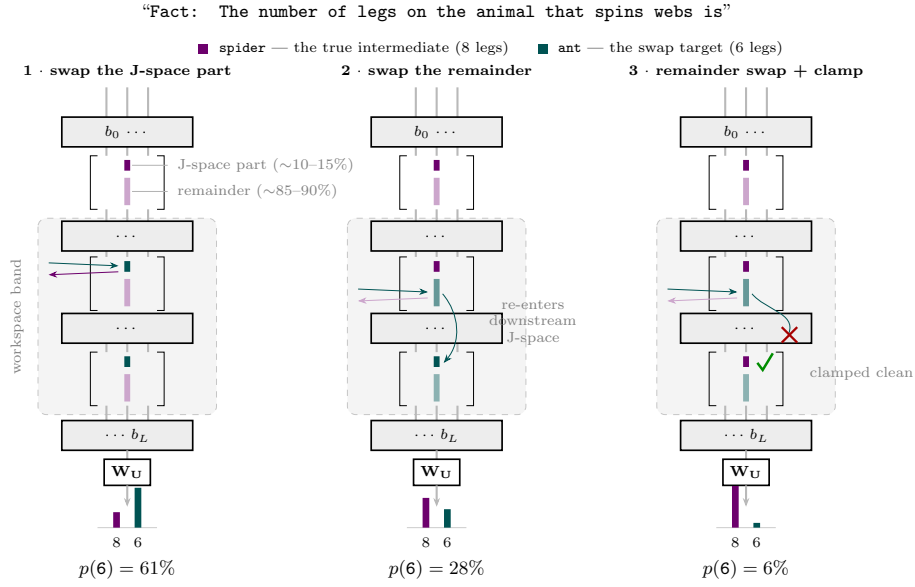


Figure 11: The three-pronged swap. The probe direction for the unspoken intermediate (**spider**) splits into a J-space part—a sparse nonnegative combination of $k=25$ lens rows, only $\sim 10\text{--}15\%$ of the probe’s variance—and the $\sim 85\text{--}90\%$ remainder. All interventions act at every layer of the workspace band, at every token position. **(1)** Swapping just the J-space part for **ant**’s flips the answer 61% of the time. **(2)** Swapping only the remainder still works 28% of the time—but only because downstream layers read the moved content and re-express it into their own J-space (teal arrow). **(3)** Same remainder swap with the downstream J-space coordinates clamped to their clean (**spider**) values: the re-entry is blocked and the effect collapses to 6%. The $\sim 10\text{--}15\%$ carries the causal freight; the $\sim 85\text{--}90\%$ moves behavior only by being re-admitted into J-space.

the object that translates internal activations to the behavioral propensities they represent. These propensities or dispositions are actually better signals for “what the model sees as mattering” in a given input—i.e. they are better reads on the model’s abstract understanding of the input, as denoted by what it is wont to say.

I would like to see the normalized dot product alignment (mean-subtracted) of $\mathcal{L}$ readout scores and $\mathcal{S}$ components as compared with raw activations h in LLMs when they’re given sets of prompts that “mean or ask the same thing”. Now this is not a trivial notion to define, but, for example “France’s capital is:” and “What is the capital of France?” are asking for the same exact answer. So are “8 + 9 =” and “When I add 8 and 9 I get: ”. In these kinds of semantically identical factual questions, does $\mathcal{L}$ or $\mathcal{S}$ alignment outpace h alignment on the layer axis? Similar classes of prompts could be composed for summarization tasks, or, interestingly, prompts that are actually verbatim identical except for some spelling or punctuation errors. Perhaps when a question is spelled poorly as compared with a perfect baseline, the model’s disposition collapses later, as earlier layers spend their time sorting out those errors.

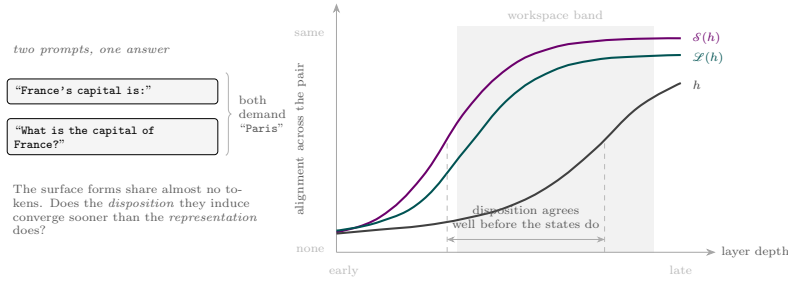


Figure 12: **A hypothesis, not a result—the curves are drawn, not measured.** Two prompts that demand the same answer share little on the surface. Plotted against depth is how closely the pair *agrees* under three readings of the same activations: the raw states h , the **J-lens** readout $\mathcal{L}(h)$, and the **J-space** decomposition $\mathcal{S}(h)$. The sketch states the conjecture this section proposes—that what a model is *disposed to say* about two paraphrases converges earlier in depth than what it *represents* about them, and that the sparse $\mathcal{S}$ support (do the two select the same tokens?) converges earliest of all. No axis carries units and no curve carries data; the shape is the claim, and testing it is the point.

These are open questions, but **J-vision** gives us the ability to ask them.

6 Causal Privilege

Calculus is the language of cause. To influence a system purposefully, to understand it to the point of intervention, one must map its elements for their causal privilege. In many systems that we build from the ground up, calculus isn’t needed to map cause through the system. But since neural networks are developed indirectly to the human mind, we have to rediscover the causal roots that took hold in the network using calculus.

The **J-lens** gave us the **J-space**, a causally privileged set of levers that

training discovered, bootstrapped, and put to use. Through it, we begin to wrest the most vexing curse of deep learning. Ceding the manual construction of a hierarchy of causal privilege, we let backpropagation—whose perceptual field and logic of influence is calculus—deliver us a coherent system to the understanding of whose logic we held no initial claim. Stunning proficiency outran stable understanding. That we’ve now found this leverage and made it legible in a system that draws such due awe is an achievement to behold—an achievement that should excite us.

The **J-space** is not an end reached, but the first revelation of the internal coherence we coaxed into emergence. Endowed with too much representational and algorithmic capacity, these systems crystallized an obvious coherence—obvious by their stunning behavior, which has already marshaled trillions of dollars of attention. For such a jungle to behave so coherently, there had to have been hierarchies formed, the effective complexity had to have collapsed; local, deterministic(, “dumb”) behaviors had to be bent *upward* in service of a higher level organization. This is causal emergence.³⁴ What better equipment with which to discover the causal structures we’ve blindly coerced than the language of cause itself?

Coloring the network with legible causal links is an old achievement. Only when the models proved to have organized hierarchically—to have allocated privilege non-uniformly on the inside as evidenced by the non-trivial success on the outside—was it time to peel the layers back and find that privilege. Random networks can be interpreted end-to-end in terms of legible cause. Only when they evidenced to have reflected our choice domain into their internal mechanism did we have—at once—the ability and *obligation* to find that reflection in the form of privileged neural circuits.

7 The Horizons of J-vision

As I wrote this document, I stepped out to pick up dinner and indulged in a whistle-along car ride. I am a good whistler, but tonight there was unusual accord between the sounds I wanted and those that were broadcast from my mouth. It stirred me to think about the control my brain was exerting over many distinct modules of my body. The physics of human whistling are far beyond my competency, and I know for sure its theorems and equations are etched nowhere in my brain. All that my brain needs to know to realize my conviction—which we won’t bother interrogating today—is a **J-lens** differentiating *whistled frequency* with respect to what I am loosely imagining as a stream of *electrical command vectors* propagating from my brain to my diaphragm, lungs, throat, lips, and tongue. The algorithm does not need realization, only $\mathbb{E}[\frac{\partial \text{Pitch}}{\partial \text{Command}}]$ does (and

³⁴ *Causal emergence* is a technical notion (Hoel, Albantakis, and Tononi, 2013): a coarse-grained *macro*-scale of a system can hold *more* causal power—a higher *effective information*—than the fine-grained *micro*-scale beneath it. It is invoked here deliberately rather than as loose metaphor: the model’s behavior is steered by a few macro-directions that training distilled from a vastly over-parameterized micro-substrate.

even this quantity is not *realized*, but baked into the machinery).

Why $\mathbb{E}$? I can whistle with similar aptitude across many different physical contexts; wet lips after a shower, dry lips on a cold winter day, laying on my back, walking on the treadmill, engaging my tongue or not, optimizing for volume or exactness...In each context, the machine to control is slightly different, but the function that describes how to change notes in any of these contexts is overwhelmingly quite similar. Whatever the realization of the whistling **J-lens** in my brain, it is an average over contexts. I can pick up whistling without having any conscious awareness of the moisture on my lips or in the air. Sure, once this information is known, there's some steering and tuning that occurs (the **J-lens** is not all-seeing, the **J-space** does not exist or act alone!), but there is a directional map of where to go in *electrical command space* to reach a desired location in *whistled frequency space*.

One more feature of the whistling lens bears mentioning here, as it relates to robustness of disposition. The map from *command space* to *frequency space* is *non-injective*, meaning it exhibits **many-to-one** relationships. There are many, many specific commands my brain can stream that will achieve the same frequency, especially when the diverse contexts above are considered. There is, then, a notion of *fibers* with respect to this map. There are directions in command space that can be traversed that will not change the corresponding output in frequency space. If I want to stay on tune and remain alive, I must breathe through my performance; my brain evidently has found invariant fibers along which it should travel in command space as the periods of respiration occur. The same idea goes for the moisture of my lips, which is constantly changing, especially when I whistle. There are circuits in the brain that can pin the required invariant—pitch—with a proficient honoring of the fibers admitted under the **J-lens**. **J-vision** gives us a new intuitive vocabulary with which to explain cybernetics—in differentiable and non-differentiable machines alike.

The mystery of how high-level mental goals filter down through mechanism into realization has always tantalized me. **J-vision** is the beginning of the answer. Anthropic's results suggested that Claude's downstream weights specifically amplify privileged **J-space** directions internally, and we have to imagine that such a privileged space (in any layer ℓ) developed in concert with other parameters (in layers $\ell' < \ell$) that learned to exert high-leverage influence on the future behavior of the model by manifesting changes in h_ℓ . But beyond a system's internal apprehension (by way of **J-vision**) of its high leverage states, we as machine learning engineers are tantalized by *our own* interventional opportunities as *external* agents. If the brain has modules akin to language models, we can imagine that higher-level systems have found their own levered hooks informed by **J-vision** as well that can enable executive steering.

J-vision is both the generalized ability to map changes in high-leverage control variables to those in downstream domains of behavior *as well as* the disposition to think about and look at hierarchical systems in this framing. **J-vision** the ethos seeks **J-vision** the practice, which fans the flames of the ethos, and so on. We've just gotten our first **J-glimpse**.